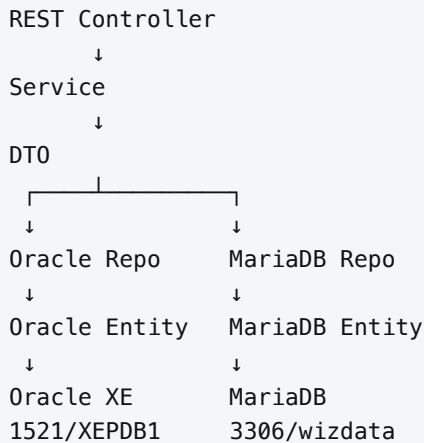


# Spring Boot Java + Kotlin Oracle + MariaDB Multi-Database 완성 교재

OracleConfig + MariaDbConfig · Entity · Repository · DTO · Service · Controller · CRUD

## 1. 전체 아키텍처



text

멀티 DB에서는 Oracle과 MariaDB에 각각 DataSource, EntityManagerFactory, TransactionManager를 구성하고 Repository 패키지를 분리하는 것이 핵심임.

## 2. 권장 프로젝트 구조

```

src/main/java(or kotlin)/com/example/multidb/
├── MultiDbApplication
├── config/
│   ├── OracleConfig
│   └── MariaDbConfig
├── oracle/
│   ├── entity/OracleMember
│   └── repository/OracleMemberRepository
├── mariadb/
│   ├── entity/MariaMember
│   └── repository/MariaMemberRepository
├── dto/MemberDto
├── service/MemberService
└── controller/MemberController
  
```

text

### 3. Maven 의존성

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.oracle.database.jdbc</groupId>
    <artifactId>ojdbc11</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.mariadb.jdbc</groupId>
    <artifactId>mariadb-java-client</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

xml

Kotlin 프로젝트는 kotlin-reflect, kotlin-stdlib-jdk8 및 Spring/JPA 관련 Kotlin 플러그인을 추가함.

### 4. application.yml

```
app:
  datasource:
    oracle:
      jdbc-url: jdbc:oracle:thin:@localhost:1521/XEPDB1
      username: devuser
      password: oracle_password
      driver-class-name: oracle.jdbc.OracleDriver

    mariadb:
      jdbc-url: jdbc:mariadb://localhost:3306/wizdata
      username: devuser
      password: mariadb_password
      driver-class-name: org.mariadb.jdbc.Driver

spring:
  jpa:
    open-in-view: false
    properties:
      hibernate:
        format_sql: true
```

yaml

### 5. Oracle + MariaDB 접속 비교 10개

| # | 항목       | Oracle                                  | MariaDB                               |
|---|----------|-----------------------------------------|---------------------------------------|
| 1 | JDBC URL | jdbc:oracle:thin:@localhost:1521/XEPDB1 | jdbc:mariadb://localhost:3306/wizdata |
| 2 | 기본 포트    | 1521                                    | 3306                                  |

| #  | 항목              | Oracle                     | MariaDB                   |
|----|-----------------|----------------------------|---------------------------|
| 3  | Driver          | oracle.jdbc.OracleDriver   | org.mariadb.jdbc.Driver   |
| 4  | 서비스/DB          | XEPDB1                     | wizdata                   |
| 5  | DataSource Bean | oracleDataSource           | mariaDataSource           |
| 6  | EMF             | oracleEntityManagerFactory | mariaEntityManagerFactory |
| 7  | Transaction     | oracleTransactionManager   | mariaTransactionManager   |
| 8  | Entity 패키지      | oracle.entity              | mariadb.entity            |
| 9  | Repository      | oracle.repository          | mariadb.repository        |
| 10 | JPA 구현          | Hibernate/JPA              | Hibernate/JPA             |

## 6. Java OracleConfig

```

@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = "com.example.multidb.oracle.repository",
    entityManagerFactoryRef = "oracleEntityManagerFactory",
    transactionManagerRef = "oracleTransactionManager"
)
public class OracleConfig {

    @Primary
    @Bean
    @ConfigurationProperties("app.datasource.oracle")
    public DataSource oracleDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Primary
    @Bean
    public LocalContainerEntityManagerFactoryBean oracleEntityManagerFactory(
        EntityManagerFactoryBuilder builder) {
        return builder
            .dataSource(oracleDataSource())
            .packages("com.example.multidb.oracle.entity")
            .persistenceUnit("oracle")
            .build();
    }

    @Primary
    @Bean
    public PlatformTransactionManager oracleTransactionManager(
        @Qualifier("oracleEntityManagerFactory")
        EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}

```

java

## 7. Java MariaDbConfig

java

```
@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = "com.example.multidb.mariadb.repository",
    entityManagerFactoryRef = "mariaEntityManagerFactory",
    transactionManagerRef = "mariaTransactionManager"
)
public class MariaDbConfig {

    @Bean
    @ConfigurationProperties("app.datasource.mariadb")
    public DataSource mariaDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean mariaEntityManagerFactory(
        EntityManagerFactoryBuilder builder,
        @Qualifier("mariaDataSource") DataSource dataSource) {
        return builder
            .dataSource(dataSource)
            .packages("com.example.multidb.mariadb.entity")
            .persistenceUnit("mariadb")
            .build();
    }

    @Bean
    public PlatformTransactionManager mariaTransactionManager(
        @Qualifier("mariaEntityManagerFactory")
        EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}
```

## 8. Kotlin OracleConfig

kotlin

```
@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = ["com.example.multidb.oracle.repository"],
    entityManagerFactoryRef = "oracleEntityManagerFactory",
    transactionManagerRef = "oracleTransactionManager"
)
class OracleConfig {

    @Primary
    @Bean
    @ConfigurationProperties("app.datasource.oracle")
    fun oracleDataSource(): DataSource =
        DataSourceBuilder.create().build()

    @Primary
    @Bean
    fun oracleEntityManagerFactory(
        builder: EntityManagerFactoryBuilder
    ): LocalContainerEntityManagerFactoryBean =
        builder
            .dataSource(oracleDataSource())
            .packages("com.example.multidb.oracle.entity")
            .persistenceUnit("oracle")
            .build()

    @Primary
    @Bean
    fun oracleTransactionManager(
        @Qualifier("oracleEntityManagerFactory") emf: EntityManagerFactory
    ): PlatformTransactionManager =
        JpaTransactionManager(emf)
}
```

## 9. Kotlin MariaDbConfig

```
@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = ["com.example.multidb.mariadb.repository"],
    entityManagerFactoryRef = "mariaEntityManagerFactory",
    transactionManagerRef = "mariaTransactionManager"
)
class MariaDbConfig {

    @Bean
    @ConfigurationProperties("app.datasource.mariadb")
    fun mariaDataSource(): DataSource =
        DataSourceBuilder.create().build()

    @Bean
    fun mariaEntityManagerFactory(
        builder: EntityManagerFactoryBuilder,
        @Qualifier("mariaDataSource") dataSource: DataSource
    ): LocalContainerEntityManagerFactoryBean =
        builder
            .dataSource(dataSource)
            .packages("com.example.multidb.mariadb.entity")
            .persistenceUnit("mariadb")
            .build()

    @Bean
    fun mariaTransactionManager(
        @Qualifier("mariaEntityManagerFactory") emf: EntityManagerFactory
    ): PlatformTransactionManager =
        JpaTransactionManager(emf)
}
```

## 10. Oracle + MariaDB 테이블 생성 SQL 10개 비교

| 테이블         | Oracle 핵심 SQL                                                  | MariaDB 핵심 SQL                                |
|-------------|----------------------------------------------------------------|-----------------------------------------------|
| MEMBER      | ID NUMBER GENERATED BY DEFAULT AS IDENTITY, NAME VARCHAR2(100) | id BIGINT AUTO_INCREMENT, name VARCHAR(100)   |
| PRODUCT     | PRICE NUMBER(15,2)                                             | price DECIMAL(15,2)                           |
| BOARD       | CONTENT CLOB                                                   | content LONGTEXT                              |
| EMPLOYEE    | SALARY NUMBER(12,2)                                            | salary DECIMAL(12,2)                          |
| CUSTOMER    | PHONE VARCHAR2(30)                                             | phone VARCHAR(30)                             |
| ORDERS      | CREATED_AT DATE DEFAULT SYSDATE                                | created_at DATETIME DEFAULT CURRENT_TIMESTAMP |
| PAYMENT     | STATUS VARCHAR2(20)                                            | status VARCHAR(20)                            |
| UPLOAD_FILE | FILE_PATH VARCHAR2(500)                                        | file_path VARCHAR(500)                        |
| CATEGORY    | NAME VARCHAR2(100)                                             | name VARCHAR(100)                             |
| ACCESS_LOG  | MESSAGE VARCHAR2(1000)                                         | message VARCHAR(1000)                         |

## 11. 대표 MEMBER 테이블 전체 SQL

---

### Oracle

```
CREATE TABLE MEMBER (  
  ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  NAME VARCHAR2(100) NOT NULL,  
  EMAIL VARCHAR2(200),  
  AGE NUMBER,  
  CREATED_AT DATE DEFAULT SYSDATE  
);
```

sql

### MariaDB

```
CREATE TABLE member (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(200),  
  age INT,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

sql

## 12. Java Entity

---

```
// OracleMember.java  
@Entity  
@Table(name="MEMBER")  
public class OracleMember {  
    @Id  
    @GeneratedValue(strategy=GenerationType.IDENTITY)  
    private Long id;  
    private String name;  
    private String email;  
    private Integer age;  
  
    public OracleMember() {}  
    // getters/setters  
}  
  
// MariaMember.java  
@Entity  
@Table(name="member")  
public class MariaMember {  
    @Id  
    @GeneratedValue(strategy=GenerationType.IDENTITY)  
    private Long id;  
    private String name;  
    private String email;  
    private Integer age;  
  
    public MariaMember() {}  
    // getters/setters  
}
```

java

## 13. Kotlin Entity

```
@Entity
@Table(name="MEMBER")
class OracleMember(
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String? = null,
    var age: Int? = null
)

@Entity
@Table(name="member")
class MariaMember(
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String? = null,
    var age: Int? = null
)
```

kotlin

## 14. Java Repository + DTO

```
public interface OracleMemberRepository
    extends JpaRepository<OracleMember, Long> {
    List<OracleMember> findByNameContaining(String name);
}

public interface MariaMemberRepository
    extends JpaRepository<MariaMember, Long> {
    List<MariaMember> findByNameContaining(String name);
}

public record MemberDto(
    Long id,
    String name,
    String email,
    Integer age
) {}
```

java

## 15. Kotlin Repository + DTO

```
interface OracleMemberRepository :  
    JpaRepository<OracleMember, Long> {  
    fun findByNameContaining(name: String): List<OracleMember>  
}  
  
interface MariaMemberRepository :  
    JpaRepository<MariaMember, Long> {  
    fun findByNameContaining(name: String): List<MariaMember>  
}  
  
data class MemberDto(  
    val id: Long?,  
    val name: String,  
    val email: String?,  
    val age: Int?  
)
```

kotlin

## 16. JPA Entity → Repository → DTO 예제 10개 비교

| #  | 기능    | Java                                       | Kotlin                                    |
|----|-------|--------------------------------------------|-------------------------------------------|
| 1  | 전체 조회 | repository.findAll()                       | repository.findAll()                      |
| 2  | ID 조회 | repository.findById(1L)                    | repository.findById(1L)                   |
| 3  | 저장    | repository.save(member)                    | repository.save(member)                   |
| 4  | 삭제    | repository.deleteById(1L)                  | repository.deleteById(1L)                 |
| 5  | 개수    | repository.count()                         | repository.count()                        |
| 6  | 이름검색  | List<Member> findByName(String name)       | fun findByName(name:String): List<Member> |
| 7  | 이메일   | Optional<Member> findByEmail(String email) | fun findByEmail(email:String): Member?    |
| 8  | 나이이상  | findByAgeGreaterThanOrEqualTo(Integer age) | findByAgeGreaterThanOrEqualTo(age:Int)    |
| 9  | 포함검색  | findByNameContaining(String name)          | findByNameContaining(name:String)         |
| 10 | 정렬    | findAll(Sort.by(DESC,"id"))                | findAll(Sort.by(DESC,"id"))               |

## 17. Java Service - 두 DB 동시 사용

```
@Service
public class MemberService {

    private final OracleMemberRepository oracleRepository;
    private final MariaMemberRepository mariaRepository;

    public MemberService(
        OracleMemberRepository oracleRepository,
        MariaMemberRepository mariaRepository) {
        this.oracleRepository = oracleRepository;
        this.mariaRepository = mariaRepository;
    }

    @Transactional("oracleTransactionManager")
    public OracleMember saveOracle(OracleMember member) {
        return oracleRepository.save(member);
    }

    @Transactional("mariaTransactionManager")
    public MariaMember saveMaria(MariaMember member) {
        return mariaRepository.save(member);
    }

    public List<MemberDto> oracleMembers() {
        return oracleRepository.findAll().stream()
            .map(m -> new MemberDto(
                m.getId(), m.getName(), m.getEmail(), m.getAge()))
            .toList();
    }

    public List<MemberDto> mariaMembers() {
        return mariaRepository.findAll().stream()
            .map(m -> new MemberDto(
                m.getId(), m.getName(), m.getEmail(), m.getAge()))
            .toList();
    }
}
```

java

## 18. Kotlin Service - 두 DB 동시 사용

```
@Service
class MemberService(
    private val oracleRepository: OracleMemberRepository,
    private val mariaRepository: MariaMemberRepository
) {
    @Transactional("oracleTransactionManager")
    fun saveOracle(member: OracleMember) =
        oracleRepository.save(member)

    @Transactional("mariaTransactionManager")
    fun saveMaria(member: MariaMember) =
        mariaRepository.save(member)

    fun oracleMembers(): List<MemberDto> =
        oracleRepository.findAll().map {
            MemberDto(it.id, it.name, it.email, it.age)
        }

    fun mariaMembers(): List<MemberDto> =
        mariaRepository.findAll().map {
            MemberDto(it.id, it.name, it.email, it.age)
        }
}
```

kotlin

## 19. Java REST Controller

```
@RestController
@RequestMapping("/api/members")
public class MemberController {

    private final MemberService service;

    public MemberController(MemberService service) {
        this.service = service;
    }

    @GetMapping("/oracle")
    public List<MemberDto> oracle() {
        return service.oracleMembers();
    }

    @GetMapping("/mariadb")
    public List<MemberDto> mariadb() {
        return service.mariaMembers();
    }
}
```

java

## 20. Kotlin REST Controller

```
@RestController
@RequestMapping("/api/members")
class MemberController(
    private val service: MemberService
) {
    @GetMapping("/oracle")
    fun oracle() = service.oracleMembers()

    @GetMapping("/mariadb")
    fun mariadb() = service.mariaMembers()
}
```

kotlin

## 21. Java ↔ Kotlin 핵심 비교

| 항목                | Java                  | Kotlin                |
|-------------------|-----------------------|-----------------------|
| DTO               | record/class          | data class            |
| Repository        | extends JpaRepository | : JpaRepository       |
| 생성자 주입            | 생성자 작성                | 주생성자로 간단히 처리          |
| Null              | Optional/@Nullable 등  | 타입에 ? 표시              |
| Entity            | getter/setter 필요      | 프로퍼티 중심               |
| Service           | 코드량 상대적으로 많음          | 간결함                   |
| Transaction       | @Transactional("...") | @Transactional("...") |
| JPA 원리            | 동일                    | 동일                    |
| Oracle/MariaDB 분리 | Config/Package 분리     | Config/Package 분리     |
| 추천                | 기존 Java 프로젝트          | 신규 간결한 Spring 프로젝트    |

## 22. 실행 및 테스트 순서

- 1) Oracle Docker 1521 및 MariaDB Docker 3306이 실행 중인지 확인함.
- 2) Oracle XEPDB1에 devuser 계정과 MEMBER 테이블을 준비함.
- 3) MariaDB wizdata에 devuser 계정과 member 테이블을 준비함.
- 4) application.yml의 IP, 사용자, 비밀번호를 실제 서버 환경에 맞춤.
- 5) Spring Boot를 실행함.
- 6) GET /api/members/oracle 로 Oracle 데이터를 확인함.
- 7) GET /api/members/mariadb 로 MariaDB 데이터를 확인함.
- 8) 저장 메서드에서는 사용할 DB에 맞는 TransactionManager를 명시함.

## 23. 실무 핵심 주의사항

- 멀티 DataSource에서는 @Primary를 한쪽(본 교재에서는 Oracle)에 명확히 지정함.

- Oracle Repository와 MariaDB Repository의 basePackages가 겹치지 않게 분리함.
- 각 EntityManagerFactory의 packages()가 해당 DB Entity 패키지만 스캔하도록 설정함.
- 한 메서드에서 Oracle과 MariaDB를 동시에 수정하는 경우 단일 로컬 @Transactional만으로 두 DB 전체의 원자성을 자동 보장하지 않음. 분산 트랜잭션이 필요한지는 별도로 설계해야 함.
- 운영 비밀번호를 소스코드나 Git 저장소에 직접 넣지 말고 환경변수/Secret 방식을 권장함.
- Oracle과 MariaDB의 IDENTITY, 날짜형, CLOB/LONGTEXT 등 SQL 차이를 확인함.

---

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER\_IP)·계정(devuser)·비밀번호(CHANGE\_ME...)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.